

# Aula 2 — Setup RN, Manual Estruturado & Unit Testing

*"Antes de automatizar UI, e o que vem antes?"*

Foco da disciplina: **React Native** (app de referência é RN)

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 28/05/2026

## Abertura — Fluxo Fork + PR (~15min)

Antes do conteúdo técnico, vamos cobrir **como entregar atividades via GitHub**.

 Slides + guia escrito no repo da disciplina:

- `slides/intro-fluxo-pr-github.pdf`
- `COMO_ENTREGAR.md` (raiz do repo)

Cobre: fork → clone → branch → commit → push → PR → link Canvas.

Termina com **demo ao vivo** num repo dummy.

| Garantir que ninguém trave na entrega da próxima atividade.

## Recap da Aula 1

- Ecossistema mobile: 3M apps, 77% abandono
- Pirâmide de Knott vs Cohn
- Tipos de teste mobile (10 categorias)
- Real device vs emulator vs cloud farm

### Atividade 1 — Análise de Cobertura entregue?

## Agenda da aula (3h30)

1. **Bloco 1 (90min)** — Setup RN + Manual estruturado (SBTM)
2. **Intervalo (15min)**
3. **Bloco 2 (90min)** — Hands-on: testes unitários RN (Jest + RNTL)
4. **Wrap-up (15min)** — Atividade 2

## Objetivos de aprendizagem

- **Configurar** ambiente React Native (Node, Metro, simulators iOS/Android)
- **Estruturar** testes manuais com SBTM (Bach)
- **Aplicar** heurísticas FEW HICCUPPS (Bolton)
- **Implementar** testes unitários RN — Jest (domain puro) + React Native Testing Library
- **Aplicar** test doubles (Meszaros) com `jest.fn` / `jest.mock`

## Setup ambiente RN — checklist

```
# Node 22 LTS (unit tests Jest rodam só com Node)
nvm install 22 && nvm use 22

# Watchman (Metro file watcher) + Cocoapods (build iOS)
brew install watchman cocoapods

# Build nativo: Xcode 16+ (iOS) e Android Studio + AVD (Android)
# necessário só pra rodar o app no simulator – unit test não precisa

# RN starter (Community CLI)
npx @react-native-community/cli@latest init MeuApp

# Verificar
node -v # v22.x
cd MeuApp && yarn test # Jest roda o teste exemplo do template
```

**Pra QA RN:** unit (Jest) = só Node. Simulator/build nativo só pra E2E (Detox/Maestro, Aulas 3–4).

## Manual testing estruturado — por que importa

Na **Atividade 1** vocês já usaram FEW HICCUPPS + SBTM pra redigir casos. Aqui **formalizamos** a técnica — não é repetição, é aprofundar.

Em mobile, **manual ainda é peça central** porque automação não cobre:

- Sensação de UX (responsividade tátil)
- Comportamento em conectividade variável
- Interação com sensores
- Permissões iOS/Android dialogs
- Comportamento em background/foreground transitions
- Acessibilidade real

**Mas manual sem estrutura é caos.** SBTM resolve.

## Session-Based Test Management (SBTM)

Proposto por **James Bach**.

**Sessão:** unidade de tempo dedicada (60–120min).

### Estrutura:

1. **Charter** — missão da sessão em 1 frase
2. **Tester** — quem executa
3. **Duração** — alvo (60min, 90min, 120min)
4. **Notas** — log do que fez, o que viu
5. **Bugs** — issues levantadas
6. **Issues** — perguntas/debates
7. **Debriefing** — discussão pós-sessão

Estrutura sem virar script. Foco com flexibilidade.



## Charter — exemplo

**CHARTER:**

Explorar fluxo de checkout do app de e-commerce  
focando em: cupons, métodos de pagamento alternativos  
e cancelamentos antes de confirmação.

Objetivo: descobrir bugs de UX/funcionais.

Duração: 90min.

| Charter dá foco. Tester explora dentro do escopo, livre pra perseguir descobertas.

## Heurísticas — FEW HICCUPPS

**Michael Bolton.** Heurística pra cobertura sistemática:

- **Familiar** — conformidade com padrões da plataforma
- **Explicit** — comportamento explicitamente especificado
- **World** — comportamento esperado pelo usuário real
- **History** — comportamento histórico do produto
- **Image** — imagem da empresa/produto
- **Comparable products** — comparáveis no mercado
- **Claims** — promessas marketing/docs
- **User desires** — desejos não declarados
- **Purpose** — propósito do produto
- **Product** — coerência interna
- **Statutes** — leis, regulações

## Whittaker tours

**James Whittaker** propõe "tours" pelo app:

- **Money tour** — onde o produto gera receita
- **Landmark tour** — features principais
- **Antisocial tour** — entradas inválidas, edge cases
- **Obsessive tour** — repetir mesma ação ad infinitum
- **All-nighter tour** — deixar app aberto 24h+
- **FedEx tour** — como dado entra e sai do app

Use como prompt para sessões SBTM.

## Bug reporting — template eficaz

TÍTULO: [iOS 17] Botão Comprar não responde após cupom expirado

PASSOS:

1. Adicionar item ao carrinho
2. Aplicar cupom expirado (CUPOM\_TESTE\_2024)
3. Tap no botão "Comprar"

ESPERADO: Mensagem "Cupom expirado, remova para continuar"

ATUAL: Botão não responde, sem feedback visual

REPRO RATE: 5/5 (sempre)

DEVICE: iPhone 14 Pro, iOS 17.5

APP VERSION: 2.3.1 (build 2310)

SCREENCAST: [link.loom.com/abc123](https://link.loom.com/abc123)

**Repro rate** é diferencial. "Aconteceu uma vez" é hipótese, não bug.

## Unit testing RN — por onde começar

Pirâmide de unit num app RN, de baixo (mais barato) pra cima:

1. **Domain puro** — funções/regras de negócio em TS, sem React, sem nativo → Jest puro, milissegundos
2. **Hooks / lógica de estado** — `@testing-library/react-hooks` ou render mínimo
3. **Componentes** — React Native Testing Library (RNTL), foco em comportamento

**Comece pelo domain.** Maior ROI, zero flaky, roda só com Node.

## Domain layer — Jest puro

Lógica de negócio isolada de React e do nativo. Teste mais barato e estável.

```
// src/domain/cart.ts
export function cartTotal(items: Item[]): number {
  return items.reduce((sum, i) => sum + i.price * i.qty, 0);
}
```

```
// src/domain/__tests__/cart.test.ts
import { cartTotal } from '../cart';

describe('cartTotal', () => {
  it('soma preço × quantidade de cada item', () => {
    const items = [
      { id: '1', price: 10, qty: 2 },
      { id: '2', price: 5, qty: 1 },
    ];
    expect(cartTotal(items)).toBe(25);
  });

  it('retorna 0 pra carrinho vazio', () => {
    expect(cartTotal([])).toBe(0);
  });
});
```

## Mockando dependências — jest.mock

Camada **data** depende de API/storage. Mocka a dependência, testa a lógica.

```
// src/data/checkout.ts
import { api } from './api';
export async function checkout(cart: Cart) {
  const res = await api.charge(cart.total);
  return { transactionId: res.id };
}

// src/data/__tests__/checkout.test.ts
import { checkout } from '../checkout';
import { api } from '../api';

jest.mock('../api'); // auto-mock do módulo
const mockedCharge = api.charge as jest.Mock;

it('chama o gateway com o total e devolve a transação', async () => {
  mockedCharge.mockResolvedValue({ id: 'tx-1' }); // stub

  const result = await checkout({ total: 100 });

  expect(result.transactionId).toBe('tx-1');
  expect(mockedCharge).toHaveBeenCalledWith(100); // spy/mock: verifica interação
```

## Componentes — React Native Testing Library

Testa **comportamento** (o que o usuário vê/faz), não implementação.

```
import { render, fireEvent, waitFor } from '@testing-library/react-native';
import LoginScreen from '../LoginScreen';

describe('LoginScreen', () => {
  it('disables submit when email is invalid', () => {
    const { getByPlaceholderText, getByText } = render(<LoginScreen />);

    fireEvent.changeText(getByPlaceholderText('Email'), 'invalid');

    expect(getByText('Entrar')).toBeDisabled();
  });

  it('calls onSubmit with valid credentials', async () => {
    const onSubmit = jest.fn();
    const { getByPlaceholderText, getByText } = render(<LoginScreen onSubmit={onSubmit} />);

    fireEvent.changeText(getByPlaceholderText('Email'), 'a@b.com');
    fireEvent.changeText(getByPlaceholderText('Senha'), 'secret');
    fireEvent.press(getByText('Entrar'));

    await waitFor(() => {
      expect(onSubmit).toHaveBeenCalledWith({ email: 'a@b.com', password: 'secret' });
    });
  });
});
```



## Test doubles (Meszaros)

| Tipo  | O que faz                                 | Quando usar                              |
|-------|-------------------------------------------|------------------------------------------|
| Dummy | Objeto qualquer, nunca usado              | Preenche parâmetro                       |
| Stub  | Retorna respostas pré-programadas         | State-based testing                      |
| Spy   | Stub + grava chamadas                     | Verificar quantas vezes algo foi chamado |
| Mock  | Verifica chamadas com expectativas        | Interaction-based testing                |
| Fake  | Implementação simplificada (in-memory DB) | Replicar dependência cara                |

Em Jest: `jest.fn()` = spy/mock · `.mockReturnValue()` = stub · módulo fake em `__mocks__` / .

| Não confunda. Stub responde, mock observa.

## Cobertura — Jest + Istanbul

```
# RN usa Istanbul por baixo do Jest
jest --coverage
open coverage/lcov-report/index.html
```

```
// jest.config.js – gate de cobertura
module.exports = {
  preset: 'react-native',
  collectCoverageFrom: ['src/**/*.{ts,tsx}', '!src/**/*.d.ts'],
  coverageThreshold: {
    global: { branches: 70, functions: 70, lines: 70, statements: 70 },
  },
};
```

| Target inicial: **70%+ em data/domain layer**. UI pode ter cobertura menor.

## Cuidado com cobertura inflada

✗ **Test que renderiza componente mas não tem assertion** → coverage 100%, valor 0.

✓ **Cobertura útil = código executado e verificado.**

▮ Mutation testing (Stryker) prova isso. Veremos na M3 (QA EAD) e na Aula 6.

## Hands-on — vamos fazer juntos (90min)

App-alvo: **mesmo app RN da disciplina de Arquitetura** (TMDB, já implementado). Hoje a gente **escreve a suíte de testes dele**.

```
git clone https://github.com/jacksonsmith/puc-iec-testes-aplicacoes-mobile.git
cd puc-iec-testes-aplicacoes-mobile/exercicios/02-setup-suite-unitaria/starter
npm install && npm test # posterUrl já passa verde
```

### Roteiro

1. `npm test` verde + ler `posterUrl.test.ts` (modelo) (10min)
2. `favoritesStore` — escrever os 6 testes juntos (25min)
3. `counterStore` — vocês fazem sozinhos (10min)
4. `isTokenError` — função pura da camada data (15min)
5. Cobertura `npm run test:coverage` → ler relatório (10min)
6. Bônus: `fetchPopularMovies` com `jest.mock` da api (10min)
7. Sessão SBTM rápida aplicada ao app (10min)

## Pitfalls comuns

- ✗ **Sleep em unit test** — anti-pattern absoluto
- ✗ **Mock de tudo** — testes acoplam à implementação, não ao comportamento
- ✗ **Test names sem nada explicativo** ( `testFunc1` ) — rename pra revelar intenção
- ✗ **Coverage como métrica única** — alta cobertura com mutation score baixo é ilusão
- ✗ **Manual testing sem charter** — vira clicar aleatório

## Atividade 2: Suíte Unitária (10 pts)

**Entrega: até 11/06.** Fork do starter + sua suíte de testes.

Entregar:

1. **favoritesStore** — 6 testes verdes (add/remove/toggle/isFavorite/clear)
2. **counterStore** — 3 testes (increment/decrement/reset)
3. **isTokenError** (data) — 5 testes
4. **Cobertura  $\geq$  70%** em `src/store` e `src/utils`
5. **Critério eliminatório:** `npm install && npm test` roda em < 15min
6. **Bônus (+1):** `MovieCard` com RNTL

Enunciado completo no repo: `exercicios/02-setup-suite-unitaria/enunciado.md`

## Leituras obrigatórias (pré-Aula 3)

- Knott — *Hands-On Mobile App Testing*, cap 5–6
- Meszaros, G. — *xUnit Test Patterns* (Addison-Wesley 2007), cap 5
- Khorikov, V. — *Unit Testing: Principles, Practices, and Patterns* (Manning 2020), cap 4
- Bach, J. — *Session-Based Test Management* (artigo seminal)

## Próxima aula (08/06)

### Aula 3 — Espresso, XCUITest e Detox

Pré-requisito:

- Atividade 2 entregue
- Apps starter buildando em iOS + Android



# Obrigado!

 [jackson.96@gmail.com](mailto:jackson.96@gmail.com)

 [linkedin.com/in/3jacksonsmith](https://linkedin.com/in/3jacksonsmith)

 [github.com/jacksonsmith](https://github.com/jacksonsmith)

**Próxima segunda, 08/06, 19:00**